MLA0303_RL_PROGRAM_4.py

```
"""
Experiment 4: Bellman Equations for an Autonomous Delivery Robot.
Value Iteration (repeated application of the Bellman optimality
equation) is used to find the minimum travel-cost path from a depot
to a delivery destination on a weighted grid.
"""

GRID_ROWS, GRID_COLS = 5, 5
DEPOT = (0, 0)
DESTINATION = (4, 4)
BLOCKED = {(1, 2), (2, 2), (3, 1)}

# Travel cost of moving INTO each cell (e.g. traffic/terrain difficulty)
COST = {(r, c): 1 for r in range(GRID_ROWS) for c in range(GRID_COLS)}
COST[(2, 3)] = 4   # heavy traffic cell
COST[(1, 4)] = 3   # rough terrain cell

DESTINATION_BONUS = 15
GAMMA = 0.95
THETA = 1e-4

ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]
DELTA = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}


def in_bounds(pos):
    r, c = pos
    return 0 <= r < GRID_ROWS and 0 <= c < GRID_COLS and pos not in BLOCKED


def next_state_reward(state, action):
    dr, dc = DELTA[action]
    nxt = (state[0] + dr, state[1] + dc)
    if not in_bounds(nxt):
        return state, -COST[state] - 5     # heavy penalty for hitting wall/blocked cell
    reward = -COST[nxt]
    if nxt == DESTINATION:
        reward += DESTINATION_BONUS
    return nxt, reward


def value_iteration():
    V = {(r, c): 0.0 for r in range(GRID_ROWS) for c in range(GRID_COLS) if (r, c) not in BLOCKED}
    sweeps = 0
    while True:
        delta = 0
        for s in list(V.keys()):
            if s == DESTINATION:
                continue
            best = float("-inf")
            for a in ACTIONS:
                s2, r = next_state_reward(s, a)
                if s2 not in V:
                    continue
                best = max(best, r + GAMMA * V[s2])
            delta = max(delta, abs(best - V[s]))
            V[s] = best
        sweeps += 1
        if delta < THETA:
            break
    return V, sweeps


def extract_policy(V):
    policy = {}
    for s in V:
        if s == DESTINATION:
            continue
        best_a, best_val = None, float("-inf")
        for a in ACTIONS:
            s2, r = next_state_reward(s, a)
            if s2 not in V:
                continue
            val = r + GAMMA * V[s2]
            if val > best_val:
                best_val, best_a = val, a
        policy[s] = best_a
    return policy


def trace_path(policy):
    state = DEPOT
    path = [state]
    total_cost = 0
    while state != DESTINATION:
        action = policy[state]
        nxt, _ = next_state_reward(state, action)
        total_cost += COST[nxt]
        state = nxt
        path.append(state)
    return path, total_cost


if __name__ == "__main__":
    V, sweeps = value_iteration()
    print(f"Bellman value iteration converged in {sweeps} sweeps.\n")
    policy = extract_policy(V)
    path, total_cost = trace_path(policy)
    print(f"Optimal minimum-cost path from {DEPOT} to {DESTINATION}:")
```

```
print(" -> ".join(str(p) for p in path))
print(f"Total travel cost: {total_cost}")
```

PROGRAM OUTPUT

```
Bellman value iteration converged in 9 sweeps.

Optimal minimum-cost path from (0, 0) to (4, 4):
(0, 0) -> (1, 0) -> (2, 0) -> (3, 0) -> (4, 0) -> (4, 1) -> (4, 2) -> (4, 3) -> (4, 4)
Total travel cost: 8
```